```python
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

st.set_page_config(page_title="DataScope | Dataset Explorer", page_icon="рџ“Љ", layout="wide")
st.title("рџ“Љ DataScope вЂ” Dataset Explorer")
st.write("Explore datasets, inspect statistics, check data quality, and create charts.")

st.sidebar.header("Choose a dataset")
choice = st.sidebar.radio("Dataset source", ["Built-in Iris dataset", "Upload a CSV file", "Use a sample CSV"])

if choice == "Built-in Iris dataset":
    iris = load_iris(as_frame=True)
    df = iris.frame
    df["species"] = df["target"].map(dict(enumerate(iris.target_names)))
    df = df.drop(columns=["target"])
    st.sidebar.caption("Source: scikit-learn's built-in Iris dataset.")
elif choice == "Upload a CSV file":
    uploaded_file = st.sidebar.file_uploader("Upload a .csv file", type=["csv"])
    if uploaded_file is None:
        st.info("Upload a CSV file in the sidebar, or select a sample dataset.")
        st.stop()
    try:
        df = pd.read_csv(uploaded_file)
    except Exception as error:
        st.error(f"Could not read this file: {error}")
        st.stop()
else:
    sample_name = st.sidebar.selectbox("Choose a sample", ["Student performance", "House prices"])
    filename = "student_performance.csv" if sample_name == "Student performance" else "house_prices.csv"
    df = pd.read_csv("datasets/" + filename)

if df.empty:
    st.warning("This dataset has no rows.")
    st.stop()

st.subheader("1. Dataset overview")
c1, c2, c3 = st.columns(3)
c1.metric("Rows", df.shape[0])
c2.metric("Columns", df.shape[1])
c3.metric("Duplicate rows", int(df.duplicated().sum()))

with st.expander("Preview dataset", expanded=True):
    st.dataframe(df.head(10), use_container_width=True)

st.subheader("2. Statistical summary")
st.dataframe(df.describe(include="all").transpose(), use_container_width=True)

st.subheader("3. Data quality")
missing = df.isna().sum()
quality = pd.DataFrame({
    "Column": df.columns,
    "Missing values": [int(missing[col]) for col in df.columns],
    "Missing (%)": [round(float(missing[col]) / len(df) * 100, 2) for col in df.columns],
    "Data type": [str(df[col].dtype) for col in df.columns],
})
st.dataframe(quality, use_container_width=True)
if int(missing.sum()) == 0:
    st.success("No missing values found.")
else:
    st.warning(f"Found {int(missing.sum())} missing cell(s).")

st.subheader("4. Create a chart")
numeric_columns = df.select_dtypes(include="number").columns.tolist()
if not numeric_columns:
    st.info("No numeric columns were found, so a numeric chart cannot be created.")
else:
    chart_type = st.selectbox("Chart type", ["Histogram", "Bar chart", "Scatter plot"])
```

```
    if chart_type in ["Histogram", "Bar chart"]:
        column = st.selectbox("Choose a column", numeric_columns)
        fig, ax = plt.subplots()
        if chart_type == "Histogram":
            ax.hist(df[column].dropna(), bins=15, edgecolor="black")
            ax.set_ylabel("Count")
        else:
            counts = df[column].dropna().value_counts().sort_index().head(20)
            ax.bar(counts.index.astype(str), counts.values)
            ax.set_ylabel("Frequency")
            ax.tick_params(axis="x", rotation=45)
        ax.set_title(f"{chart_type} of {column}")
        ax.set_xlabel(column)
        st.pyplot(fig)
        plt.close(fig)
    else:
        if len(numeric_columns) < 2:
            st.info("A scatter plot needs at least two numeric columns.")
        else:
            x_col = st.selectbox("X-axis", numeric_columns, index=0)
            y_options = [col for col in numeric_columns if col != x_col]
            y_col = st.selectbox("Y-axis", y_options)
            fig, ax = plt.subplots()
            ax.scatter(df[x_col], df[y_col], alpha=0.7)
            ax.set_xlabel(x_col)
            ax.set_ylabel(y_col)
            ax.set_title(f"{y_col} vs {x_col}")
            st.pyplot(fig)
            plt.close(fig)

st.subheader("5. Quick insights")
st.write(f"- The dataset contains **{df.shape[0]} rows** and **{df.shape[1]} columns**.")
st.write(f"- It contains **{int(df.duplicated().sum())} duplicate rows**.")
st.write(f"- It contains **{int(missing.sum())} missing values** in total.")
if numeric_columns:
    chosen = st.selectbox("Get an insight about a numeric column", numeric_columns,
key="insight_column")
    average = df[chosen].mean()
    if pd.notna(average):
        st.write(f"- The average value of **{chosen}** is **{average:.2f}**.")

st.subheader("6. Download the dataset")
st.download_button("Download current dataset as CSV", data=df.to_csv(index=False).encode("utf-8"),
                   file_name="datascope_dataset.csv", mime="text/csv")
st.caption("Educational project: check a dataset's source and context before drawing conclusions.")
```